
디자인부터 성능최적화까지 역할을 가리지 않고 문제를 해결하는 프론트엔드 개발자



안윤선 | Frontend Developer

SmartPod 검교정 시스템

IoT 기기의 임계값 감지 및 이상 진단 정확도를 검증하는 웹 기반 검교정 시스템

📅 기간 2025.01 ~ 2025.03

👥 인원 기획 1, 개발 2

🔧 기술 React, TypeScript, Styled-components, Zustand

📌 개발 배경

콜드체인 물류에서 IoT 데이터로거가 측정하는 온도, 습도, 충격 값의 정확도는 핵심 신뢰 요소입니다. 센서 오차를 주기적으로 검증하지 않으면 실제 온도와의 괴리가 발생할 수 있어, 임계값 이상 진단의 정확도를 검증하는 시스템이 필요했습니다.

☰ 담당 역할

디자인

- 와이어프레임 및 비주얼 UI 디자인
- 정상/이상 상태 구분 UI 설계
- 측정 데이터 테이블 설계

개발

- 타입 기반 디자인 시스템 구축
- 기기 등록, 임계값 설정, 충격 데이터 측정, 결과 확인 등 주요 화면 개발

정상·이상 상태를 한눈에 인지할 수 있는 UI 설계

○ 상황

- IoT 측정값의 정상·이상 상태를 직관적으로 구분할 수 있는 UI 설계 필요

✓ 수행

1 요구사항 문서 기반 UI/UX 디자인

기기 등록 → 임계값 설정 → 측정 → 이상 진단 결과 확인까지의 흐름을 화면 단위로 구조화

2 정상/이상 상태 구분 UI 및 측정 데이터 테이블 설계

단순 텍스트 라벨링만으로는 다수의 측정값을 한눈에 스캔하기 어렵다고 판단해 색상·배지를 활용한 상태 표시 체계를 설계

다량의 측정값을 순차적으로 비교·검토해야 하는 업무 특성을 고려해, 데이터를 테이블 형태로 구조화하여 한눈에 확인할 수 있도록 설계

3 Figma 와이어프레임 제작

설계한 상태 구분 체계를 실제 화면에 적용해보며 정보 위계와 시인성 검증

➤ 결과

- 디자이너 리소스 없이 일정 내 화면 설계 완료
- 상태 구분이 명확한 UI로 측정 데이터 이상 여부를 빠르게 인지 가능한 화면 설계 완료
- 개발 착수 전 화면 구조 및 상태별 UI 규칙이 명확히 정의되어 개발 단계에서의 재설계 비용 감소

UI 일관성 확보를 위한 타입 기반 디자인 시스템 구축

○ 상황

- 디자인 시스템의 부재로 UI 파편화 및 개발 생산성 저하

✓ 수행

1 타입 기반 디자인 토큰 설계

허용되지 않는 값 입력 시 컴파일 단계에서 에러가 발생하도록 색상/폰트/사이즈를 TypeScript 객체로 정의
타입 안전성과 리팩토링 용이성을 위해 채택

2 variant 패턴 적용

props에 따라 분기하는 삼항 연산자 대신 사전 정의된 옵션만 입력받도록 설계해 디자인 가이드라인 위반을 원천 차단하고,
컴포넌트 내부 로직 가독성 향상

➤ 결과

- 신규 화면의 UI 요소 중 공통 컴포넌트 적용 비율 **70%**
- 디자인 시스템 도입 전 평균 5시간 → 도입 후 평균 3시간으로 UI 개발 시간 **약 40% 단축**
- 스타일 파편화·가이드라인 위반 이슈 0건

SmartPod B2B 고객사 포털

고객사별 배송 상태와 IoT 기기 데이터를 조회·모니터링하는 웹 서비스

📅 기간 2024.06 ~ 2024.09

👥 인원 기획 1, 디자인 1, 개발 2

🔗 기술 Vue3, JavaScript, TailwindCSS, Pinia

📄 개발 배경

고객사가 콜드체인 배송 중인 화물의 상태를 현장에서 실시간으로 확인해야 하는 경우가 많아 PC보다 모바일 웹뷰 환경에서의 접근성이 중요했습니다. 다양한 모바일 기종에서도 안정적인 화면과 빠른 로딩 속도를 제공하는 것이 핵심 과제였습니다.

☰ 담당 역할

개발

- 배송 현황 및 IoT 기기 데이터 조회·모니터링 화면 개발
- 온도·습도·충격 이력 데이터 차트로 시각화
- 모바일 웹뷰 환경의 반응형 UI 구현 및 다양한 디바이스 대응
- 번들 최적화 및 지연 로딩을 통한 초기 로딩 성능 개선

QA

- Android·iOS 기종별 반응형 UI 및 모바일 웹뷰 동작 검증
- 주요 사용자 시나리오 기반 기능 테스트 및 이슈 수정

모바일 웹뷰 대응 및 번들 최적화로 초기 로딩 개선

○ 상황

- 모바일 웹뷰 환경 내 기종별 차이로 레이아웃 간섭 발생
- 단일 번들 구조로 인한 초기 로딩 속도 저하

✓ 수행

1 반응형 레이아웃 설계

고정 px 기반으로 다양한 화면 크기 대응이 불가능해 그리드·상대 단위 구조로 전환
CSS env()로 기종별 노치·홈 바 안전 영역까지 자동 대응

2 번들 경량화

단일 번들 구조에서는 미방문 페이지 코드까지 다운로드되어 초기 로딩 저하
→ 라우트 단위 동적 import()로 페이지별 번들 분리, 초기 진입 화면의 비노출 컴포넌트는 defineAsyncComponent로 지연 로딩

↗ 결과

- Chrome DevTools Dimensions 기준 주요 Android/iOS 기종에서 일관된 UI 제공
- Chrome DevTools Network 기준 초기 JS 다운로드 용량 1.24MB → 1.05MB로 약 **15% 감소**

SmartPod 메인 서비스

IoT 기반 콜드체인(GPS · 온도 · 습도 · 충격) 실시간 관제 플랫폼

📅 기간 2023.04 ~ 2025.07

👥 인원 기획 2, 디자인 1, 개발 4

🔧 기술 Vue3, JavaScript, Tailwind CSS, Pinia, vue-i18n

📄 개발 배경

콜드체인 물류 과정에서 IoT 기기가 수집하는 GPS·온도·습도·충격 데이터를 실시간으로 모니터링하고, 배송 및 기기 상태를 통합 관리하기 위한 관제 서비스가 필요했습니다.

☰ 담당 역할

개발

- Tailwind CSS 마이그레이션을 통한 스타일 코드 일관성 및 유지보수성 개선
- vue-i18n 기반 한국어·영어 다국어 지원 기능 개발
- locale 초기화 순서 개선을 통한 언어 전환 화면 깜빡임 해결

vue-i18n 기반 다국어 지원 및 언어 전환 화면 깜빡임 개선

○ 상황

- 한국어로 제공되던 서비스에 영어를 추가하여 다국어 지원 필요
- 저장된 언어 설정을 렌더링 이후 적용하면서, 페이지 진입 시 기본 언어인 한국어가 잠시 노출된 뒤 선택한 언어로 전환되는 화면 깜빡임(locale flickering) 발생

✓ 수행

1 vue-i18n 기반 다국어 구조 구현

ko.json, en.json으로 UI 텍스트를 구조화하고 vue-i18n을 활용해 현재 locale에 맞는 번역 텍스트를 제공

2 언어 설정 적용 시점 분석 및 깜빡임 원인 파악

기본 locale로 먼저 렌더링된 후 저장된 언어가 적용되면서 발생하는 리렌더링이 화면 깜빡임의 원인임을 파악

3 최초 렌더링 전 locale 초기화로 화면 깜빡임 개선

createI18n 초기화 전에 저장된 locale을 확인해 초기값으로 설정하여, 최초 렌더링부터 선택한 언어가 적용되도록 개선

➤ 결과

- 한국어·영어 다국어 지원을 위한 번역 리소스 구조 및 언어 전환 기능 구현
- 기본 언어가 순간적으로 노출된 후 선택 언어로 변경되던 locale flickering 현상 개선
- 최초 렌더링 이전에 locale을 결정하도록 초기화 순서를 개선하여 불필요한 언어 전환 리렌더링 제거

가면(가상면접) AI

사용자가 업로드한 이력서·포트폴리오를 기반으로 AI가 맞춤형 질문을 생성하고, 시선 추적 기술로 태도를 분석해 종합 리포트를 제공하는 AI 모의 면접 시뮬레이터

📅 **기간** 2026.02 ~ 2026.08

👥 **인원** 프론트엔드 3, 백엔드 3

🔗 **기술** Next.js, TypeScript, TailwindCSS, TanStack Query, Zustand

📌 개발 배경

면접 스터디의 일정·공간 제약과 혼자서는 면접 태도를 객관적으로 파악하기 어렵다는 문제를 해결하고자 했습니다. 이를 해결하기 위해 이력서·자기소개서·포트폴리오 기반의 맞춤형 질문으로 실전 면접을 연습하고, 답변 내용과 시선 처리 등 비언어적 요소까지 피드백 받을 수 있는 AI 모의면접 서비스를 기획했습니다.

☰ 담당 역할

기획

- 서비스 기획 리딩
- AI 모의면접 사용자 플로우 설계

디자인

- Figma 기반 주요 화면 UI 디자인

개발

- 모의 면접 생성부터 진행·종료까지 면접 전체 사용자 흐름 개발
- GA4 데이터 기반 사용자 유입·이탈 분석 백오피스 대시보드 구현

GA4 데이터 기반 이탈 분석 및 질문 생성 프로세스 최적화

○ 상황

- 온보딩 단계별 사용자 이탈 지점을 파악할 데이터 부재
- LLM 연산 특성상 질문 생성에 7~10초의 대기 시간 발생

✓ 수행

1 백오피스 유입/이탈 대시보드 구현

GA4, Google Analytics Data API, recharts로 커스텀 대시보드 구현, 마이크 권한 단계의 높은 이탈률과 원인 확인

2 질문 생성 API 호출 시점 전진 배치

트리거를 앞당겨 AI 연산을 백그라운드에서 미리 시작

3 TanStack Query 기반 조건부 Polling 설계

특정 시점에만 질문 목록 조회를 활성화하고 질문 생성 완료 시 자동 종료되도록 구현

➤ 결과

- 7~10초의 대기시간을 환경설정 단계에 흡수시켜 체감 대기시간을 감축하고 온보딩 이탈 개선
- 고비용 LLM 교체 대신 프론트엔드 로직 수정만으로 성능 문제 해결

미디어 업로드 파이프라인 구축

○ 상황

- 영상 데이터 서버 직접 수신 시 네트워크 대역폭 및 I/O 부하 우려

✓ 수행

1 질문 단위 분산 업로드 로직 설계 및 구현

Presigned URL로 S3에 업로드하되, 답변 종료 시점마다 분할 전송하여 트래픽 부하 분산 및 중단 시 데이터 보존

2 면접 이어하기 로직 구현

URL 쿼리스트링으로 재진입 여부를 식별하고, 답변 여부 플래그를 기준으로 미답변 질문만 필터링하여 이어서 진행

3 비정상 종료 대응 로직 구현

브라우저 종료 및 뒤로가기 시 sendBeacon으로 중단 상태 전송, 실패 시 fetch + keepalive로 폴백 처리

➤ 결과

- 세션당 약 140MB 영상을 서버 경유 없이 S3로 직접 전송하도록 설계하여 서버 트래픽 부하 **90% 절감**
- 정상/비정상 종료 모든 케이스에서 데이터 손실 없이 이어하기 가능